

NITROGEN: An Open Foundation Model for Generalist Gaming Agents

Loïc Magne^{1*}, Anas Awadalla^{1 2*}, Guanzhi Wang^{1 3*†}
 Yinzhen Xu¹, Joshua Belofsky⁴, Fengyuan Hu¹, Joohwan Kim¹
 Ludwig Schmidt², Georgia Gkioxari³, Jan Kautz¹
 Yisong Yue^{3†}, Yejin Choi^{1 2†}, Yuke Zhu^{1 5†}, Linxi Fan^{1†}

¹ NVIDIA, ² Stanford, ³ Caltech, ⁴ UChicago, ⁵ UT Austin

* Co-lead, † Co-advise

<https://nitrogen.minedojo.org>

Abstract

We introduce NITROGEN, a vision-action foundation model for generalist gaming agents that is trained on 40,000 hours of gameplay videos across more than 1,000 games. We scale embodied agents through three key ingredients: 1) an internet-scale video-action dataset constructed by automatically extracting player actions from publicly available gameplay videos, 2) a multi-game benchmark environment that can measure cross-game generalization, and 3) a unified vision-action model trained with large-scale behavior cloning. NITROGEN exhibits competence across diverse domains, including combat encounters in 3D action games, high-precision control in 2D platformers, and exploration in procedurally generated worlds. When fine-tuned, pre-training transfers effectively to unseen games, achieving up to 52% relative improvement in task success rates over models trained from scratch. We release the dataset, evaluation suite, and model weights to advance research on generalist embodied agents.

1. Introduction

Building generally capable embodied agents that can operate in unknown environments has long been considered a holy grail of AI research. While computer vision and large language models (LLMs) have achieved this generalization through large-scale pre-training on internet data [10, 15, 16, 47], comparable progress in embodied AI has been impeded by the lack of large, diverse, and labeled action datasets. Video games present an ideal domain for advancing embodied AI since they offer visually rich interactive environments and tasks that span a wide range of complexities and temporal horizons. However, prior approaches face substantial limitations. **LLM-based methods** exploit either (1) hand-crafted programmatic APIs to expose in-

ternal game states and control agents [59–61] or (2) complicated perception modules for textual information extraction and object detection [51]. They enable complex task-solving but require complicated domain-specific design and tuning. **Reinforcement learning** has achieved superhuman performance in individual games such as StarCraft II and Dota 2, but these agents are narrow, costly to train, and depend on specialized simulators rarely available for arbitrary games [5, 37, 38, 49, 58]. **Behavior-cloning approaches** based on pixel observations have relied on expensive-to-collect demonstrations, constraining training to only a few game titles due to prohibitive data collection costs [3, 46]. To date, there has been little progress on developing open-source frameworks that can support the training and evaluation of generalist gaming agents, further hindering progress in this direction.

To address these limitations, we introduce NITROGEN, an open foundation model for video game environments trained on 40,000 hours of publicly available internet videos covering more than 1,000 games. We make three major contributions (Figure 1):

1. Internet-scale dataset of action-labeled videos. We propose to use a new source of data from publicly available videos where content creators overlay their input commands in real time. We train an annotation model to extract frame-level actions with high accuracy, removing the need for costly manual data collection and capturing a wide spectrum of real player behaviors. Using this approach, we curate a dataset of 40,000 hours of video spanning more than 1,000 games, providing diverse demonstrations for large-scale training.

2. Multi-task multi-game evaluation suite. To assess generalization in realistic settings, we design benchmark environments that comprise 30 tasks of varied complexity from 10 commercial games, covering diverse challenges such as combat, navigation, decision-making, platforming, exploration, and puzzle-solving. This benchmark reflects

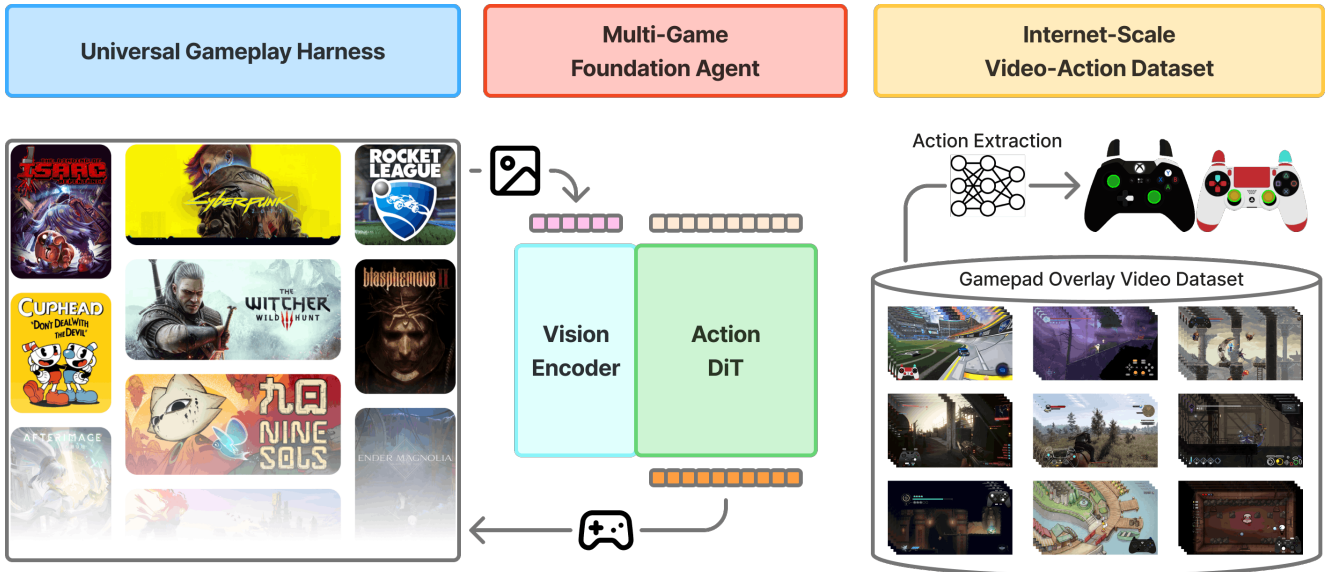


Figure 1. **NITROGEN overview.** NITROGEN consists of three main components: (1) **Multi-game foundation agent (center)** - a generalist vision-action model that takes in game observations and generates gamepad actions, enabling zero-shot gameplay across multiple titles and serving as a foundation for fine-tuning on new games; (2) **Universal gameplay harness (left)** - an environment wrapper that allows any commercial game to be controlled through a Gymnasium API; and (3) **Internet-scale dataset (right)** - the largest and most diverse open-source gaming dataset curated from 40,000 hours of publicly available gaming videos, spanning more than 1,000 games with extracted action labels.

the demands of modern game environments, where agents must learn to adapt across heterogeneous mechanics and objectives. We provide a universal Gymnasium API [56] for our evaluation suite that allows users to wrap any game to test diverse agent capabilities. This API is what we refer to as the **universal gameplay harness** in Figure 1.

3. Large-scale behavior-cloning pre-training. To demonstrate the feasibility and benefits of internet-scale pre-training, we train a vision-action transformer model on our dataset. We demonstrate strong results on our benchmark suite, validating our end-to-end pipeline and showing that it is possible to train a strong multi-game policy using only noisy internet data. We show the benefits of behavior-cloning pre-training by post-training our base model on games not seen during training. The model fine-tuned from the pre-trained NITROGEN weights shows up to 52% relative improvement in success rates over the model trained from scratch, given a fixed data and compute budget.

We will open-source the NITROGEN dataset, simulator, and pre-trained weights upon acceptance of the paper. We envision NITROGEN as a foundational resource that will enable the research community to accelerate progress toward building more generalist embodied agents, fostering new algorithms, model architectures, and applications in this emerging area.

2. Approach

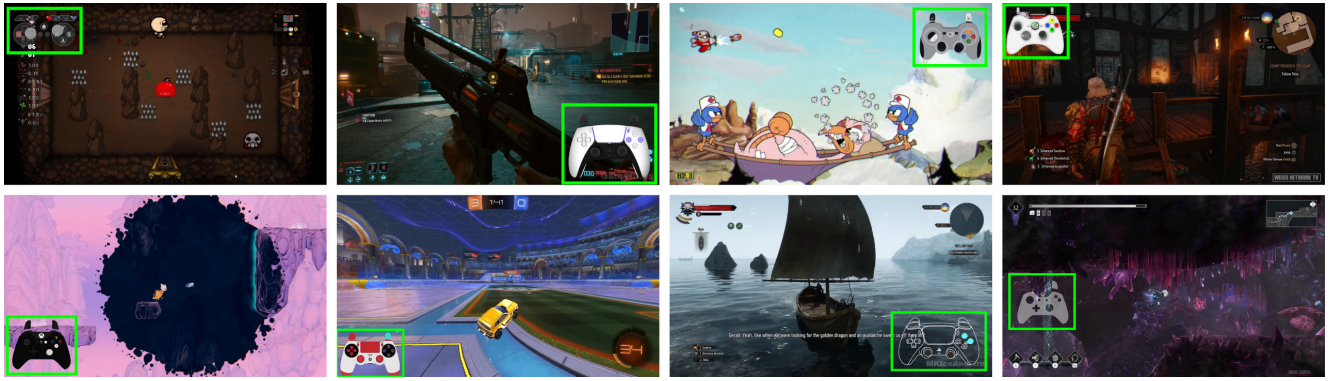
NITROGEN consists of three novel components: (1) an internet-scale video dataset with action labels, (2) a multi-

game benchmark with a Gymnasium environment wrapper, and (3) a vision-action model pre-trained through large-scale behavior cloning. In this section, we provide details of each component.

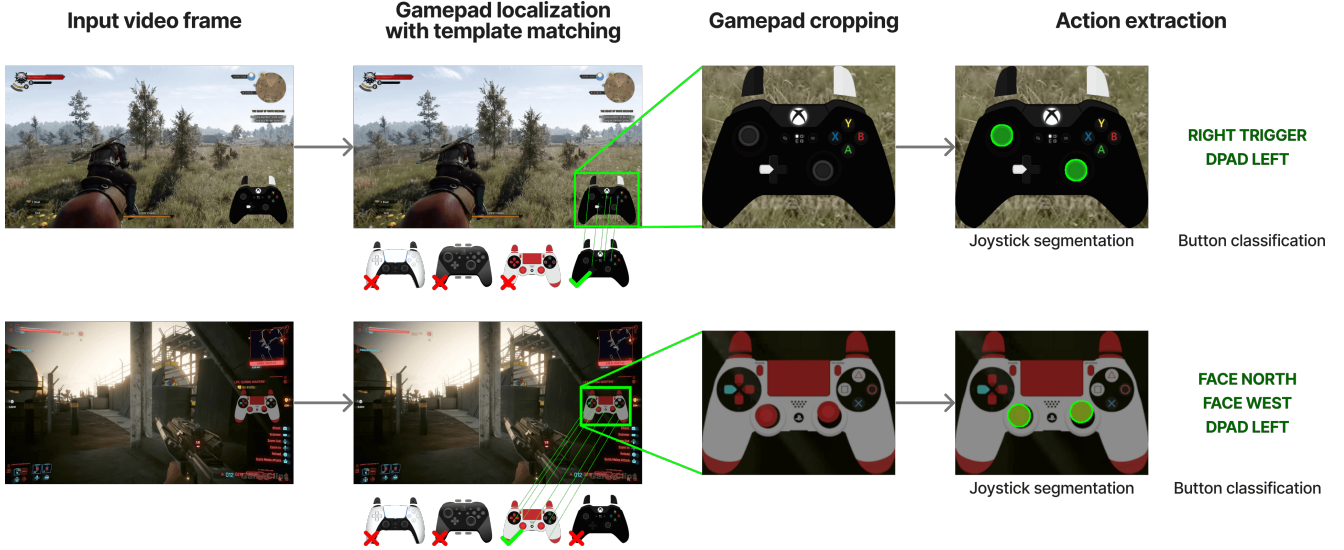
2.1. Internet-scale multi-game video-action dataset

Annotation challenge. A central challenge in training policies from internet videos is recovering the corresponding actions, since most gameplay recordings typically do not include the player’s inputs. We address this limitation by using a novel source of publicly available videos in which such labels can be recovered. These videos feature *input overlay* software that displays a real-time visualization of the player’s actions, typically as a 2D image of a gamepad in a corner of the screen with pressed buttons highlighted (Figure 2a).

Dataset curation. Although input overlays appear in only a fraction of online gameplay videos, they occur frequently enough to enable the construction of a large-scale dataset. We collect 71,000 hours of raw video containing gamepad overlays. While input overlay software was originally used primarily within the speedrunning community, its use has since expanded to many action games and among both expert and casual players. To avoid over-representation of any single title, we use a combination of keyword-based searches and curation guided by content diversity, ensuring coverage across games, genres, and skill levels. This approach balances casual and competitive play styles while maintaining broad genre representation. Figure 3 shows



(a) Examples of gamepad overlay videos.



(b) Action extraction pipeline.

Figure 2. **Video-action dataset pipeline overview.** We extract actions from on-screen displays which show the gamepad actions of the player in real-time; called “input overlays”. **(a) Dataset curation.** We collect publicly available videos displaying a “gamepad overlay”. The diversity of these overlays presents significant challenges, as gamepads vary widely across content creators in controller types (e.g., Xbox, PlayStation, or others), transparency levels, and visual artifacts introduced by video compression. **(b) Action extraction.** For each collected video, we localize the gamepad by sampling 25 frames and running **keypoint matching** against a curated set of templates using SIFT and XFeat features. We use the template-matching results to localize and crop the gamepad region from each video. A **hybrid classification-segmentation network** is then trained to predict joystick positions and button states from the cropped controller images, enabling accurate reconstruction of player inputs.

the distribution of gameplay hours by title and genre. The dataset covers more than 1,000 unique games, making it the largest labeled video-action dataset for video games to date. It contains 38,739 videos from 818 different content creators, with an average video duration of 1 hour and 50 minutes.

Action extraction. We extract player inputs from gameplay videos through a three-stage pipeline: (1) template matching to locate and crop the gamepad overlay, (2) gamepad action parsing using a fine-tuned segmentation model, and (3) quality filtering to ensure accurate and meaningful data.

Stage 1: Template matching. To locate gamepad overlays

within gameplay videos, we apply template matching using a curated set of approximately 300 common controller templates. For each video, we sample 25 frames and perform feature matching with SIFT [32] and XFeat [45] against all curated templates. We estimate an affine transformation from the paired keypoints and require at least 20 inliers for a match to be considered valid. We then extract the region with the highest matching score, which defines the gamepad location for subsequent processing. Figure 2b shows examples of successful match.

Stage 2: Gamepad action parsing. We parse controller states using a fine-tuned SegFormer [65] segmentation model that processes pairs of consecutive frames. The

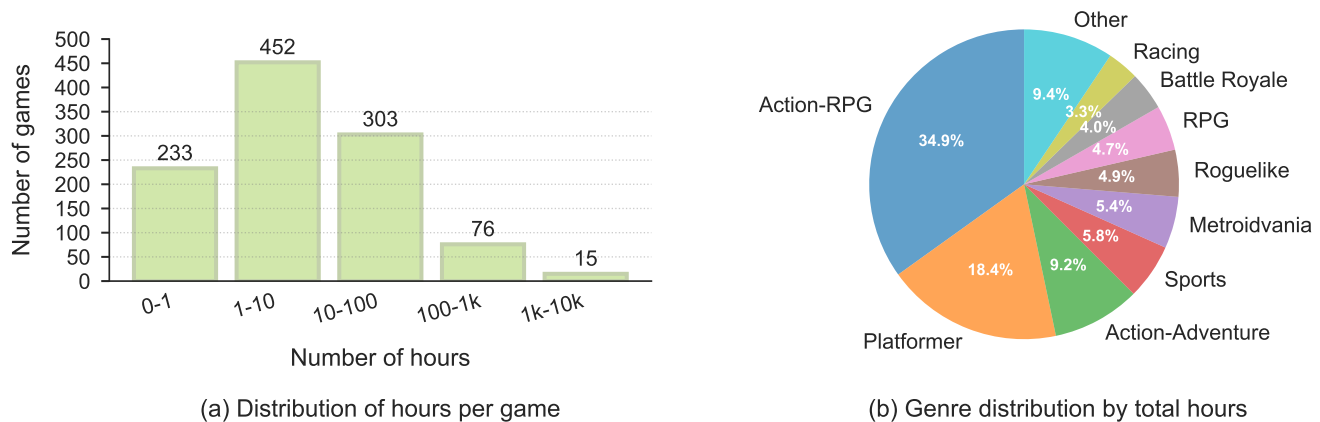


Figure 3. **Distribution of the NITROGEN dataset across games and genres.** After filtering, the NITROGEN dataset contains 40,000 hours of gameplay videos spanning more than 1,000 games. (a) Hours per game shows broad coverage, with 846 games having over one hour of data, 91 games with over 100 hours, and 15 games exceeding 1,000 hours each. (b) Genre distribution reveals Action-RPG games are most common (34.9% of total hours), followed by Platformer (18.4%) and Action-Adventure (9.2%) games, with the remainder distributed across seven genres.

model takes two consecutive frames as input (concatenated along the spatial dimension) to capture short-term temporal dynamics. It outputs a segmentation mask to localize joystick positions on a discrete 11×11 grid, and binary button states (Figure 2b). Empirically, we find that estimating joystick positions via segmentation masks significantly outperforms direct regression of joystick coordinates.

We train the annotation model using synthetic data generated by sampling frames from the NITROGEN training set and programmatically overlaying controller templates using the Open Joystick Display¹, Input Overlay², and GamePad Viewer³ software. For each template, we produce multiple frames with random button states and joystick positions, yielding 8M labeled frames. To simulate real-world visual artifacts, we vary overlay opacity, controller size, and video compression, generating multiple variants per frame. We train the action parsing SegFormer model using the AdamW optimizer [28, 31] with a learning rate of 0.0001, linear learning rate decay, weight decay of 0.1, and a batch size of 256.

At inference, we compute precise joystick positions by detecting contours for each joystick over the entire video. To estimate the center position of each joystick, we average positions from all frames where the joystick is classified as centered in the 11×11 discrete grid. We then normalize the positions to the range $[-1.0, 1.0]$ using the 99th percentile of absolute x and y values over the video to reduce the influence of outliers.

Stage 3: Quality filtering. The final stage applies targeted filtering strategies to ensure high-quality data. Dur-

ing training, we observe that using the raw 71,000 hours of data leads to the model over-predicting the null action as noted in VPT [3]. To avoid that, we discard segments based on action density: we only keep chunks where at least 50% of the timesteps have non-zero button or joystick actions, resulting in 55% of the data being kept. For all gameplay videos, we mask the on-screen controller to prevent models from exploiting it as a shortcut.

2.2. Evaluation suite

Universal harness for any game title. Many research environments provide a Gymnasium API [56] that enables programmatic control of the simulation. To bring this capability to commercial video games, which typically lack such an interface, we develop a universal gameplay harness that can wrap any game title with a Gymnasium API for model development. The library intercepts the game engine’s system clock to control simulation time, enabling frame-by-frame interaction without modifying game code. This approach works with any title that uses the system clock for physics and interactions, which is a common practice in game development. We leave real-time or asynchronous deployment to future work. Frequent pausing and resuming during gameplay could potentially affect the game’s physics engine in unknown ways, we verify that this process does not alter games’ physics and behaviors (see supplementary materials for more details).

Unified observation and action space. Using this simulator, we introduce a multi-game, multi-task benchmark with a shared interface across all titles. Observations are single RGB frames. Actions consist of a standardized 16-dimensional binary vector for gamepad buttons (4 d-pad buttons, 4 face buttons, 2 shoulders, 2 triggers, 2 joystick thumb buttons, start, back) plus a 4-dimensional continuous

¹<https://github.com/AkikoKumagara/open-joystick-display>

²<https://github.com/univrsal/input-overlay>

³<https://beta.gamepadviewer.com/>

Defeating an enemy camp using attacks, rolls, bombs, and magical signs



Puzzle solving in procedurally generated levels



First-person shooting at distant enemies with precise aiming



Precise platform-to-platform jumps



2D platforming to avoid enemies and collect a coin



Fast aerial maneuver to accurately hit the ball



Boss fight with reactive dodging, attacking, and jumping



Figure 4. **In-game rollouts.** We show NITROGEN performing tasks in diverse 2D and 3D environments. These tasks can take from a few seconds to a few minutes to perform. Some of them include memorization, while others are performed in procedurally generated worlds and require the model to adapt.

vector for joystick positions. Unlike prior work that defines game or task-specific action spaces [3, 21], this unified layout facilitates direct policy transfer across diverse games.

Diverse evaluation tasks. The evaluation suite provides a universal framework for multi-game policies, covering 10 games and 30 tasks across varied visual styles and genres. It includes five 2D games, including three side-scrollers and two top-down roguelikes with procedurally generated levels, and five 3D games, consisting of two open-world games, two combat-focused action RPGs, and one sports title.

Tasks are distributed across three categories: 11 combat tasks (boss fights, enemy encounters), 10 navigation tasks (reaching specific locations, traversing environments), and 9 game-specific tasks (unique mechanics particular to in-

dividual games). Each task has clearly defined start and goal states, with attempts typically lasting a few minutes, though human players may require several hours of repeated attempts to succeed. We select tasks where the initial visual state provides sufficient context to elicit correct behavior, leaving language-conditioned specifications to future work. Success rates are measured through human evaluation.

2.3. NITROGEN foundation model

Architecture. Building on recent advances in generative modeling and robotics, NITROGEN employs flow matching [30] to generate chunks of future actions conditioned on visual observations. The architecture is adapted from GR00T N1 [6] with the language and state encoders removed, and a single action head. RGB inputs at 256×256 resolution

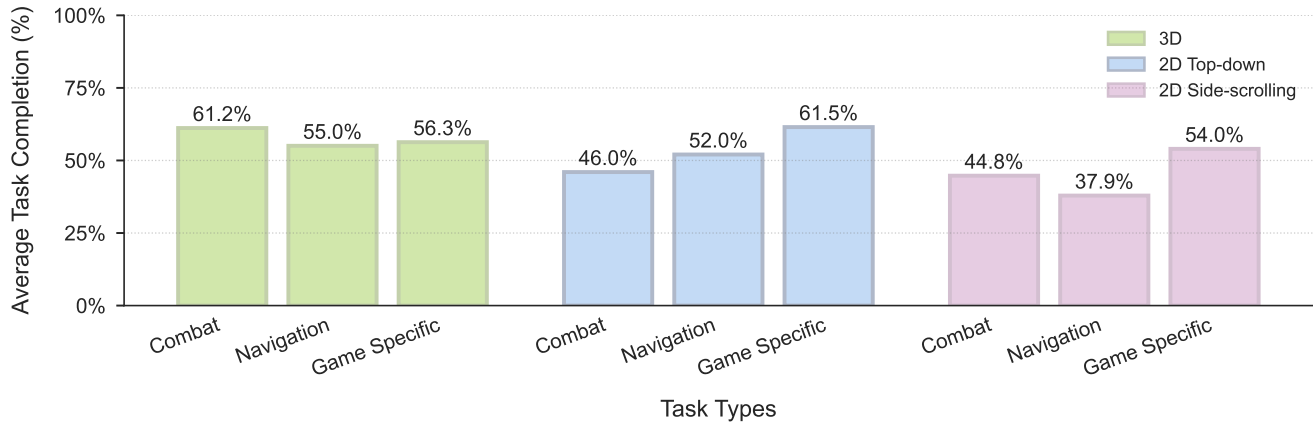


Figure 5. **NITROGEN 500M pre-training results across different games.** We evaluate NITROGEN after behavior-cloning pre-training. The model is not fine-tuned for specific games. For each game, we measure the average task completion rate on 3 tasks with 5 rollouts per task. Despite being trained on a very noisy internet dataset, NITROGEN is able to perform non-trivial tasks over games with different visual styles (3D, 2D top-down, 2D side-scrolling) and genres (platformer, action-RPG, roguelike, etc.).

are encoded using a SigLIP 2 vision transformer [57], producing 256 image tokens per frame. Actions are generated with a diffusion transformer (DiT) [42] that outputs multiple actions per forward pass. Noisy action chunks are first encoded by an MLP into one action token per timestep, then processed through several DiT blocks consisting of alternating self-attention and cross-attention layers. Cross-attention conditions action generation on the encoded frame tokens. The final action tokens are decoded into continuous action vectors using an MLP applied independently across the time dimension. Full mathematical details are provided in supplementary materials.

Design choices. Although the model can condition on multiple frames, we find no benefit from using more than one past frame, even with increased temporal gaps. This is likely because the initial state of these action games already provides sufficient context to elicit the appropriate behavior. We instead use a single context frame and generate 16-action chunks, which improves temporal consistency compared to single-action generation.

Training and inference. We train NITROGEN using the standard conditional flow-matching objective [7, 30], applied to 16-action chunks, with one 256×256 frame of context. Inference follows the corresponding denoising process with $k = 16$ steps.

During training, we apply the following image augmentations: random brightness, contrast, saturation and hue, random rotation between -5 and 5 degrees, and random crops. We train all models using AdamW [28, 31] optimizer with a weight decay of 0.001. We use a warmup-stable-decay (WSD) schedule [64], which allows us to train for longer without a fixed training budget, with a constant learning rate phase of 0.0001. Following [42], we maintain an exponential moving average (EMA) of model weights during training with a decay of 0.9999. All our results are

obtained with the EMA weights, which we find consistently outperform the non-EMA weights.

3. Experiments

Performance of the gamepad action extraction model.

To evaluate our action extraction pipeline, we construct a benchmark dataset by recording gameplay from six video games using OBS⁴, with randomized opacity, gamepad size, and gamepad type to mimic real-world conditions. We record ground-truth controller inputs at each frame and compare them with the extracted actions. We measure joystick accuracy with the R^2 score and button accuracy per frame. We achieve an average R^2 of 0.84 for joystick positions and an average button accuracy of 0.96 across the most popular controller families. Full results per controller families can be found in the supplementary materials.

NITROGEN demonstrates strong capabilities across a wide range of games. We train a single model on the entire dataset from Section 2.1. Without further fine-tuning, NITROGEN achieves non-trivial success rates across many games and tasks. Figure 5 summarizes the main results. We observe that NITROGEN performs well both on tasks that can be memorized and on tasks that require zero-shot generalization. For example, some games feature fixed layouts that the model may have partially encountered during training, while others employ procedural generation that ensures each playthrough is unique. We do not find significant differences in performance between these two categories, suggesting that NITROGEN can both leverage memorization and adapt to unseen scenarios.

This result validates that it is possible to train a robust policy using only noisy internet-scale data. The dataset

⁴Open Broadcaster Software: <https://obsproject.com/>; Input recording tool: <https://github.com/loicmagne/input-rec>

includes several sources of noise that could hinder training: **(a) actions are not strictly ground truth**, since input overlay software introduces small delays, and parsing adds further inaccuracies; **(b) video frames often contain creator-specific artifacts** such as livestream chats, subscribe prompts, or progress trackers; and **(c) controller configurations vary across players**, differences in sensitivity settings or custom button mappings can change the semantic meaning of the same input. Despite these challenges, Figure 5 shows that large-scale pre-training yields a robust multi-game policy.

NITROGEN pre-training improves downstream fine-tuning on unseen environments. We evaluate transfer learning by pre-training NITROGEN on the full dataset except for a held-out game, then fine-tuning on this game with a limited amount of data. We compare this fine-tuned model with an identical architecture trained from scratch using the same data and compute budget. Results are shown in Figure 7. We study two representative games with different visual styles and genres: an isometric roguelike and a 3D action-RPG.

The effectiveness of pre-training varies by game type and task category. Across different data quantities, fine-tuning achieves an average relative improvement of 10% on the isometric roguelike, whereas the 3D action-RPG shows a 25% average relative improvement. This difference likely stems from better representation of 3D action-RPGs in the training distribution, while the isometric roguelike has gameplay mechanics and visual style that are less common in the training data.

Furthermore, pre-training benefits are not uniform across task types. On the 3D action-RPG, generic tasks such as combat (52% relative improvement) and navigation (25% relative improvement) benefit substantially from pre-training, while game-specific tasks show only marginal gains (5% relative improvement). This suggests that NITROGEN effectively learns transferable skills for common gameplay patterns, but game-specific mechanics still require targeted training on the new environment.

NITROGEN is a strong candidate for RL fine-tuning

We conduct a self-imitation [39] experiment to illustrate that NITROGEN is amenable to RL-style fine-tuning. We focus on the first boss of a 2D metroidvania game, where the base model can damage but not consistently defeat the boss. A successful attempt takes ~ 2 minutes for a human. Since the boss behavior is non-deterministic, this task cannot be simply memorized.

We roll out the pre-trained NITROGEN model on this task and manually filter the best trajectories to form a self-imitation dataset. We then compare several settings: (1) the base NITROGEN checkpoint, (2) NITROGEN fine-tuned for 50K steps on the original BC data for this 2D game only, and (3) NITROGEN fine-tuned for 50K steps on a 50/50

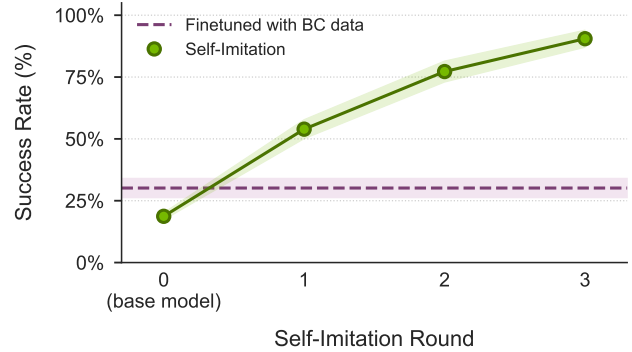


Figure 6. **Self-imitation learning rapidly improves task success rate.** We fine-tune NITROGEN on the first boss of a 2D metroidvania using rounds of self-imitation, where the best rollouts from the current model are mixed back into the training data. Starting from an 18.7% base success rate, fine-tuning on the original BC data only reaches 30.1%, while one round of self-imitation jumps to 53.9%. After three rounds, the model achieves 90.5% success rate, frequently defeating the boss.

mixture of the original data and self-imitation trajectories, from one to three rounds of self-imitation.

The base model achieves an 18.7% success rate. Fine-tuning on the original dataset alone increases this to 30.1%, while a single round of self-imitation boosts success rate to 53.9% (Fig. 6), demonstrating that the gains from self-imitation cannot be attributed simply to additional exposure to task-specific data. After three rounds of self-imitation, the model reaches 90.5% success, frequently defeating the boss. These results suggest that NITROGEN is a strong candidate for RL-style adaptation directly from pixels.

4. Limitations and future work

Scope and design limitations. NITROGEN is limited to being a fast-reacting system-1 sensory model. It is not designed to play games end-to-end, plan over long horizons or follow language instructions. It can thus only play for a few seconds, when the visual context is clear enough to predict a next action. It is also not designed for robust zero-shot gameplay on unseen games. Zero-shot capabilities would require further research, such as enabling the model to learn a game’s physics and button semantics in context.

We develop NITROGEN aiming for it to serve as a foundation for future generalist agent development, where post-training for language-following and reinforcement learning can be applied to enhance capabilities and improve success rates.

Dataset bias. While diverse, our data collection method still restricts the types of games included in our dataset. The data distribution of the NITROGEN dataset is biased toward action games (Figure 3), and games that are typically played with a gamepad. Keyboard-only games or those that involve

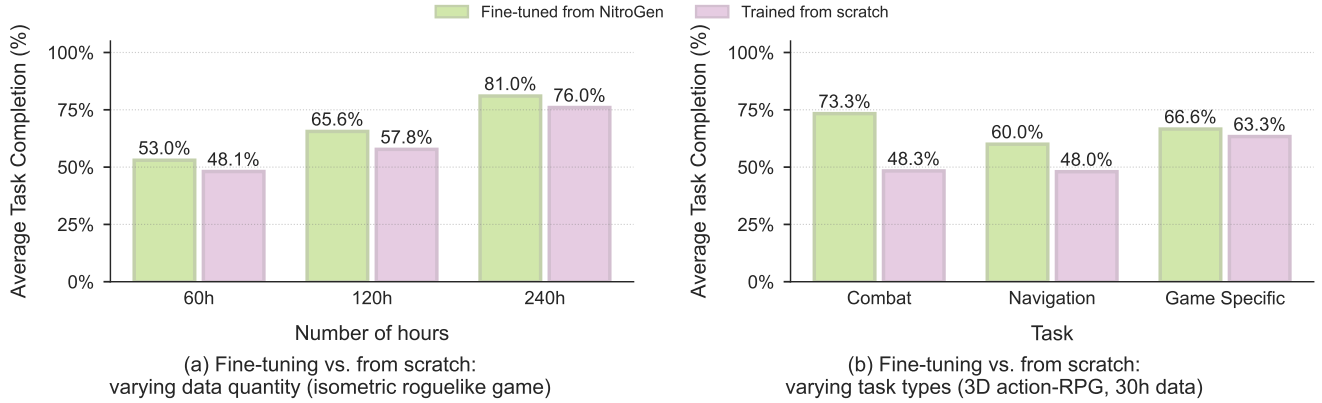


Figure 7. **Post-training experiments: NITROGEN pre-training improves downstream agents in unseen environments.** We pre-train NITROGEN on the dataset described in Section 2.1, holding out one game. We then fine-tune the pre-trained checkpoint on the held-out game and compare the results with a model trained from scratch using the same architecture, data and compute budget. (a) When varying data quantity, task-completion rate scales with dataset size, and fine-tuning achieves on average a 10% relative improvement in task-completion rate. (b) When varying task type in the low-data regime (30h), fine-tuning achieves up to 52% relative improvement in task-completion rate.

complex manipulation are less represented in the dataset. This bias may limit the agent’s ability to generalize to genres like strategy or simulation games that rely more on planning and keyboard input.

5. Related works

Gaming agents. Video games have long been testbeds for AI, spanning both classical and learning-based approaches. Prior to deep learning, game-playing AI relied on search and planning, as in GVGAI, Mario AI, and Ms. Pac-Man [24, 26, 33, 44, 54, 55], with methods such as MCTS, A*, and evolutionary algorithms [43, 44, 54]. These approaches offer complementary strengths to deep RL and imitation learning, particularly in model-based reasoning [4, 38, 55].

Reinforcement learning achieved landmark successes from Atari (DQN) to AlphaGo, AlphaStar, and OpenAI Five [5, 37, 38, 49, 58], but relies on engineered rewards and simulators. Vision-based methods such as Dreamer 3 [22] still require dedicated simulators. Other approaches use LLMs for high-level reasoning with structured APIs (e.g., Voyager, Cradle) [51, 60], while behavior cloning approaches learn directly from pixels or states, including MineRL [21], VPT [3], SIMA [46], GATO [48], Dreamer 4 [23], Lumine [52], and Farhang et al. [20]. They rely on datasets bootstrapped from human or RL-generated data, whereas NITROGEN scales behavior cloning to internet data across hundreds of games. Game-TARS [62] is a concurrent work that also trains a multi-game agent, combining contractor and multimodal reasoning data totalling over 20,000 hours.

Embodied foundation models. Embodied foundation models follow either hierarchical or end-to-end approaches. Hierarchical methods pair pre-trained LLMs or VLMs with

low-level policies [1, 17, 25, 29, 50] treating the foundation models as black-boxes. Vision-Language-Action (VLA) models [6, 8, 9, 13, 27, 53, 63] instead train policies end-to-end on embodied data, though generalizing across tasks and embodiments remains challenging. NITROGEN differs by discarding language conditioning and focusing on scalable vision-action mapping using diverse gameplay data.

Large-scale action datasets. While vision and NLP benefit from large labeled datasets, embodied AI lacks scalable action-labeled data. Gaming datasets like MineRL [21] provide limited coverage, while MineDojo [19] scales video data without action labels. VPT [3] annotates 70,000 hours via inverse dynamics but is limited to Minecraft. Other work seeks to infer latent actions from videos [11, 18, 41, 66], though scalability is unclear. In robotics, teleoperation datasets include Roboturk [34–36], ALOHA [2], TeleMoMa [14], Open X-Embodiment [40], and AgiBot World [12], but remain costly and limited in scale and diversity. NITROGEN introduces a scalable alternative by leveraging input overlay software, which naturally provides action labels in publicly available videos.

6. Conclusion

In this work, we introduce NITROGEN, an approach to scale up foundation pre-training for video game agents and demonstrate how internet pre-training can yield a generalist policy. We leverage a new source of publicly available data to build an internet-scale video-action dataset, and empirically demonstrate its effectiveness by successfully training a multi-game policy. NITROGEN shows positive signs of generalization in fine-tuning experiments. By lowering the barrier to train agents on new environments, NITROGEN serves as a starting point to develop more powerful and general-purpose agents.

References

- [1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022. 8
- [2] Jorge Aldaco, Travis Armstrong, Robert Baruch, Jeff Bingham, Sanky Chan, Kenneth Draper, Debidatta Dwibedi, Chelsea Finn, Pete Florence, Spencer Goodrich, et al. Aloha 2: An enhanced low-cost hardware for bimanual teleoperation. *arXiv preprint arXiv:2405.02292*, 2024. 8
- [3] Bowen Baker, Ilge Akkaya, Peter Zhokov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampe-dro, and Jeff Clune. Video pretraining (vpt): Learning to act by watching unlabeled online videos. *Advances in Neural Information Processing Systems*, 35:24639–24654, 2022. 1, 4, 5, 8
- [4] Marc G Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of artificial intelligence research*, 47:253–279, 2013. 8
- [5] Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Debiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, et al. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*, 2019. 1, 8
- [6] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. 5, 8, 1
- [7] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control, 2024. 6, 1
- [8] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. 8
- [9] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022. 8
- [10] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakan-tan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020. 1
- [11] Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*, 2024. 8
- [12] Qingwen Bu, Jisong Cai, Li Chen, Xiuqi Cui, Yan Ding, Siyuan Feng, Shenyuan Gao, Xindong He, Xuan Hu, Xu Huang, et al. Agibot world colosseum: A large-scale manipulation platform for scalable and intelligent embodied systems. *arXiv preprint arXiv:2503.06669*, 2025. 8
- [13] Chi-Lam Cheang, Guangzeng Chen, Ya Jing, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Hongtao Wu, Jiafeng Xu, Yichu Yang, et al. Gr-2: A generative video-language-action model with web-scale knowledge for robot manipulation. *arXiv preprint arXiv:2410.06158*, 2024. 8
- [14] Shivin Dass, Wensi Ai, Yuqian Jiang, Samik Singh, Jiaheng Hu, Ruohan Zhang, Peter Stone, Ben Abbatematteo, and Roberto Martín-Martín. Telemoma: A modular and versatile teleoperation system for mobile manipulation. *arXiv preprint arXiv:2403.07869*, 2024. 8
- [15] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional trans-formers for language understanding, 2019. 1
- [16] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Syl-vain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021. 1
- [17] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. Palm-e: An embodied multimodal language model. In *arXiv preprint arXiv:2303.03378*, 2023. 8
- [18] Ashley Edwards, Himanshu Sahni, Yannick Schroecker, and Charles Isbell. Imitating latent policies from observation. In *International conference on machine learning*, pages 1755–1763. PMLR, 2019. 8
- [19] Linxi Fan, Guanzhi Wang, Yunfan Jiang, Ajay Mandlekar, Yuncong Yang, Haoyi Zhu, Andrew Tang, De-An Huang, Yuke Zhu, and Anima Anandkumar. Minedojo: Building open-ended embodied agents with internet-scale knowledge. *Advances in Neural Information Processing Systems*, 35: 18343–18362, 2022. 8
- [20] Alexander R Farhang, Brendan Mulcahy, Daniel Holden, Iain Matthews, and Yisong Yue. Humanlike behavior in a third-person shooter with imitation learning. In *2024 IEEE Conference on Games (CoG)*, pages 1–4. IEEE, 2024. 8
- [21] William H Guss, Brandon Houghton, Nicholay Topin, Phillip Wang, Cayden Codell, Manuela Veloso, and Ruslan Salakhutdinov. Minerl: A large-scale dataset of minecraft demonstrations. *arXiv preprint arXiv:1907.13440*, 2019. 5, 8

- [22] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023. 8
- [23] Danijar Hafner, Wilson Yan, and Timothy Lillicrap. Training agents inside of scalable world models, 2025. 8
- [24] Philip Hingston. A new design for a turing test for bots. In *Proceedings of the 2010 IEEE Conference on Computational Intelligence and Games*, pages 345–350. IEEE, 2010. 8
- [25] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022. 8
- [26] Sergey Karakovskiy and Julian Togelius. The mario ai benchmark and competitions. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):55–67, 2012. 8
- [27] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024. 8
- [28] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. 4, 6
- [29] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500. IEEE, 2023. 8
- [30] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022. 5, 6
- [31] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017. 4, 6
- [32] David G Lowe. Distinctive image features from scale-invariant keypoints. *International journal of computer vision*, 60:91–110, 2004. 3
- [33] Simon M Lucas. Ms pac-man competition. *ACM SIGEVOlution*, 2(4):37–38, 2007. 8
- [34] Ajay Mandlekar, Yuke Zhu, Animesh Garg, Jonathan Booher, Max Spero, Albert Tung, Julian Gao, John Emons, Anchit Gupta, Emre Orbay, et al. Roboturk: A crowdsourcing platform for robotic skill learning through imitation. In *Conference on Robot Learning*, pages 879–893. PMLR, 2018. 8
- [35] Ajay Mandlekar, Jonathan Booher, Max Spero, Albert Tung, Anchit Gupta, Yuke Zhu, Animesh Garg, Silvio Savarese, and Li Fei-Fei. Scaling robot supervision to hundreds of hours with roboturk: Robotic manipulation dataset through human reasoning and dexterity. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1048–1055. IEEE, 2019.
- [36] Ajay Mandlekar, Danfei Xu, Roberto Martín-Martín, Yuke Zhu, Li Fei-Fei, and Silvio Savarese. Human-in-the-loop imitation learning using remote teleoperation. *arXiv preprint arXiv:2012.06733*, 2020. 8
- [37] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013. 1, 8
- [38] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015. 1, 8
- [39] Junhyuk Oh, Yijie Guo, Satinder Singh, and Honglak Lee. Self-imitation learning, 2018. 7
- [40] Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Poo-ley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE, 2024. 8
- [41] J Parker-Holder, P Ball, J Bruce, V Dasagi, K Holshemer, C Kaplanis, A Moufarek, G Scully, J Shar, J Shi, et al. Genie 2: A large-scale foundation world model. URL: <https://deepmind.google/discover/blog/genie-2-a-large-scale-foundation-world-model>, 2024. 8
- [42] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4195–4205, 2023. 6
- [43] Diego Perez, Miguel Nicolau, Michael O’Neill, and Anthony Brabazon. Evolving behaviour trees for the mario ai competition using grammatical evolution. In *European Conference on the Applications of Evolutionary Computation*, pages 123–132. Springer, 2011. 8
- [44] Diego Perez-Liebana, Spyridon Samothrakis, Julian Togelius, Tom Schaul, and Simon Lucas. General video game ai: Competition, challenges and opportunities. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016. 8
- [45] Guilherme Potje, Felipe Cadar, André Araujo, Renato Martins, and Erickson R Nascimento. Xfeat: Accelerated features for lightweight image matching. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2682–2691, 2024. 3
- [46] Maria Abi Raad, Arun Ahuja, Catarina Barros, Frederic Besse, Andrew Bolt, Adrian Bolton, Bethanie Brownfield, Gavin Buttimore, Max Cant, Sarah Chakera, et al. Scaling instructable agents across many simulated worlds. *arXiv preprint arXiv:2404.10179*, 2024. 1, 8
- [47] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, 2021. 1
- [48] Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gomez Colmenarejo, Alexander Novikov, Gabriel Barth-Maron, Mai Gimenez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, Tom Eccles, Jake Bruce, Ali Razavi, Ashley Edwards, Nicolas Heess, Yutian Chen, Raia Hadsell, Oriol

- Vinyals, Mahyar Bordbar, and Nando de Freitas. A generalist agent, 2022. 8
- [49] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016. 1, 8
- [50] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11523–11530. IEEE, 2023. 8
- [51] Weihao Tan, Wentao Zhang, Xinrun Xu, Haochong Xia, Ziluo Ding, Boyu Li, Bohan Zhou, Junpeng Yue, Jiechuan Jiang, Yewen Li, et al. Cradle: Empowering foundation agents towards general computer control. *arXiv preprint arXiv:2403.03186*, 2024. 1, 8
- [52] Weihao Tan, Xiangyang Li, Yunhao Fang, Heyuan Yao, Shi Yan, Hao Luo, Tenglong Ao, Huihui Li, Hongbin Ren, Bairen Yi, Yujia Qin, Bo An, Libin Liu, and Guang Shi. Lumine: An open recipe for building generalist agents in 3d open worlds, 2025. 8
- [53] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024. 8
- [54] Julian Togelius, Noor Shaker, Sergey Karakovskiy, and Georgios N Yannakakis. The mario ai championship 2009–2012. *Ai magazine*, 34(3):89–92, 2013. 8
- [55] Ruben Rodriguez Torrado, Philip Bontrager, Julian Togelius, Jialin Liu, and Diego Perez-Liebana. Deep reinforcement learning for general video game ai. In *2018 IEEE conference on computational intelligence and games (CIG)*, pages 1–8. IEEE, 2018. 8
- [56] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. 2, 4
- [57] Michael Tschannen, Alexey Gritsenko, Xiao Wang, Muhammad Ferjad Naeem, Ibrahim Alabdulmohsin, Nikhil Parthasarathy, Talfan Evans, Lucas Beyer, Ye Xia, Basil Mustafa, et al. Siglip 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *arXiv preprint arXiv:2502.14786*, 2025. 6
- [58] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *nature*, 575(7782):350–354, 2019. 1, 8
- [59] Ryan Vulum, Sudha Rao, Michael Xu, Gabriel DesGarnnes, Chris Brockett, Benjamin Van Durme, Olivia Deng, Akanksha Malhotra, and Bill Dolan. Craft an iron sword: Dynamically generating interactive game characters by prompting large language models tuned on code. In *Proceedings of the 3rd Wordplay: When Language Meets Games Workshop (Wordplay 2022)*, pages 25–43, Seattle, United States, 2022. Association for Computational Linguistics. 1
- [60] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. 8
- [61] Zihao Wang, Shaofei Cai, Guanzhou Chen, Anji Liu, Xiaojian Ma, and Yitao Liang. Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents, 2024. 1
- [62] Zihao Wang, Xujing Li, Yining Ye, Junjie Fang, Haoming Wang, Longxiang Liu, Shihao Liang, Juntong Lu, Zhiyong Wu, Jiazhan Feng, Wanjuan Zhong, Zili Li, Yu Wang, Yu Miao, Bo Zhou, Yuanfan Li, Hao Wang, Zhongkai Zhao, Faming Wu, Zhengxuan Jiang, Weihao Tan, Heyuan Yao, Shi Yan, Xiangyang Li, Yitao Liang, Yujia Qin, and Guang Shi. Game-tars: Pretrained foundation models for scalable generalist multimodal game agents, 2025. 8
- [63] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 2025. 8
- [64] Kaiyue Wen, Zhiyuan Li, Jason Wang, David Hall, Percy Liang, and Tengyu Ma. Understanding warmup-stable-decay learning rates: A river valley loss landscape perspective. *arXiv preprint arXiv:2410.05192*, 2024. 6
- [65] Enze Xie, Wenhao Wang, Zhiding Yu, Anima Anandkumar, José Manuel Álvarez, and Ping Luo. Segformer: Simple and efficient design for semantic segmentation with transformers. In *Neural Information Processing Systems*, 2021. 3
- [66] Seonghyeon Ye, Joel Jang, Byeongguk Jeon, Sejun Joo, Jianwei Yang, Baolin Peng, Ajay Mandlekar, Reuben Tan, Yu-Wei Chao, Bill Yuchen Lin, et al. Latent action pretraining from videos. *arXiv preprint arXiv:2410.11758*, 2024. 8